BIBLIOTECA PROVA ESAME

TRACCIA ES:

La libreria BuonaLettura gestisce l'approvvigionamento e la vendita dei libri mediante una tabella che per ogni libro riporta: codice, titolo, autori, numero copie disponibili, numero copie minimo di scorta. I clienti della libreria effettuano le loro richieste inviando il codice del libro e il numero di copie da ordinare. Progettare una soluzione al problema di gestione delle richieste che fornisca in output:

- Per ogni richiesta tratta un messaggio di richiesta accettata o rifiutata
- Il codice del libro, titolo e autori della richiesta evasa con il maggior numero di copie.
- I codici di tutti i libri le cui copie disponibili sono in esaurimento e la quantità da riordinare per mantenere il numero minimo di scorta.
- La tabella aggiornata.

ANALISI

- **Chiarifica**:
  Il programma deve gestire l'approvvigionamento della vendita dei libri tramite una tabella, tabella che sarà nota ed implementata dal programmatore il quale deciderà la sua grandezza e quanti libri conterrà, supponendo che esistano dei libri noti al programmatore, il programma deve memorizzare i dati di ogni libro, i quali dati sono il codice del libro, il quale si presuppone sia un intero, il titolo sotto forma di stringa, gli autori che si supponga venga memorizzato solamente il primo, le copie disponibili nella libreria, il quale valore è un intero assieme a quello di scorta nel magazzino.
  Si imposta il nome e l'autore attraverso le funzioni stringa definite da libreria esterna.
  Durante le richieste dell'utente si gestiscono varie casistiche. Per ogni richiesta si tratta un messaggio di richiesta con esito positivo se il libro è presente o non esiste con un esito negativo. Si presuppone che si analizza una richiesta per volta.
  Per ogni richiesta verrà conservato il numero di copie richiesto in modo tale da salvare e memorizzare i dati della richiesta che è stata evasa con il maggior numero di copie.
  Si ipotizza che venga mostrato solo uno dei dati dei libri se 2 o più libri hanno la stessa richiesta massima del numero di copie.
  Il programma controlla se le copie sono in esaurimento, ovvero se il valore della richiesta è inferiore alle copie presenti. Aggiornare la tabella dopo ogni richiesta effettuata con successo.
- **Dati**:
  - **DATI INPUT:**

	VARIABILI	TIPO	VINCOLI
Numero libri	n	intero	>0
Tabella libri: - Codice libro - autore - titolo - numero copie disponibili - libri in scorta	Lib array di record: Lib[i].codice Lib[i].autore Lib[i].titolo Lib[i].copie Lib[i].scorta	intero stringa stringa intero intero	<MAXLENGH <MAXLENGH >=0 >0
Codice del libro	cod_libro	stringa	<MAXLENGTH
Numero copie richieste	N_copie_richieste	intero	>0

- DATI OUTPUT:

	VARIABILI	TIPO	VINCOLI
Messaggio accettazione/rifiuto			
Libro più richiesto	Libromax: record: - Codice - Autore - Titolo - Numero copie - Scorta		

	VARIABILI	TIPO	VINCOLI
Tabella aggiornata	Lib array di record:	intero	<MAXLENGH
	Lib[i].codice		<MAXLENGH
		stringa	>=0
	Lib[i].autore	stringa	
	Lib[i].titolo	intero	
	Lib[i].copie		>0
		intero	
	Lib[i].scorta		
Codice di tutti i libri più richiesto			

- **DATI DI LAVORO:**

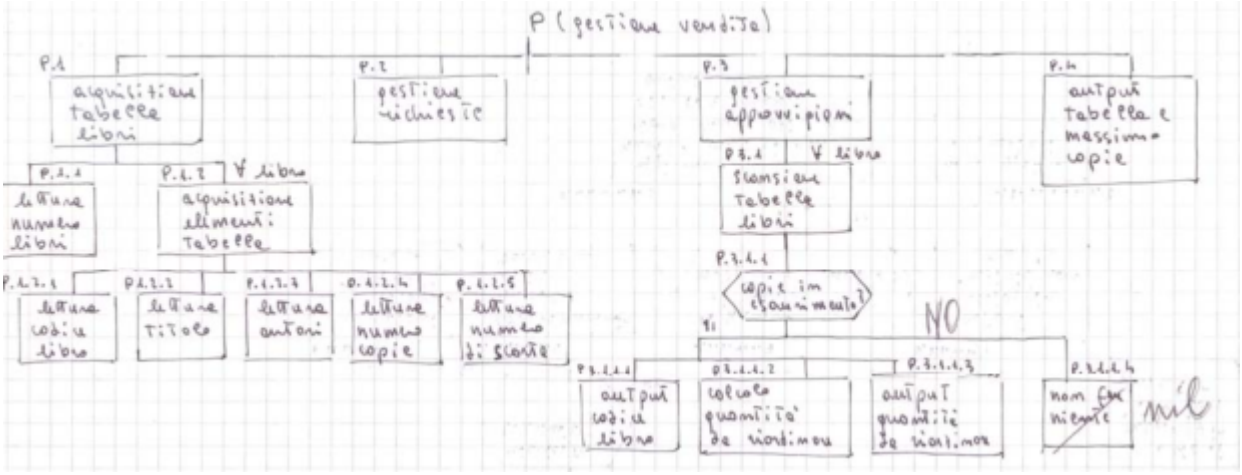
	VARIABILI	TIPO	VICNOLI
Contatore	i	intero	a seconda dei casi >0
Contatore ciclo 2	j	intero	a seconda dei casi >0
Sentinella per i controlli	flag	bool (short)	0 o 1

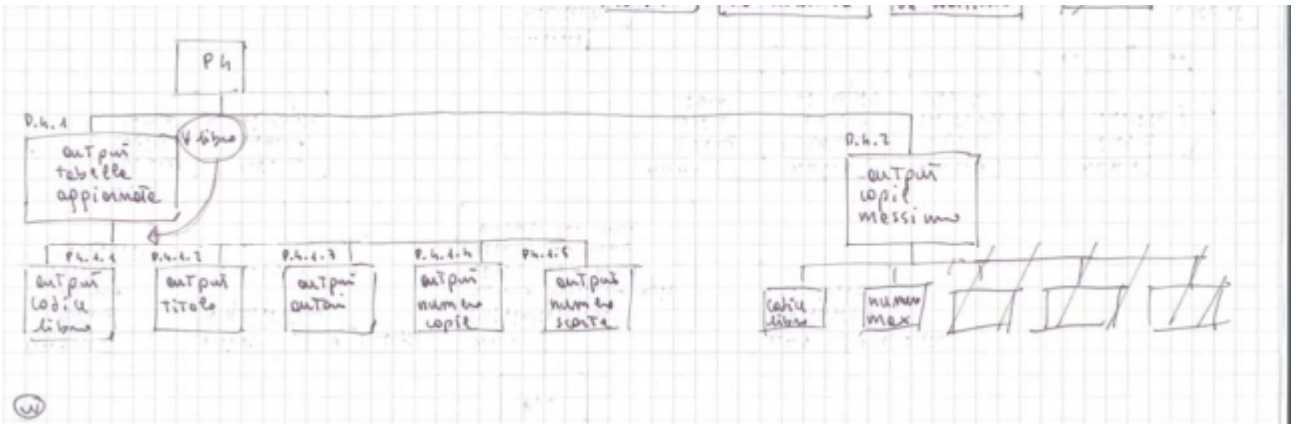
Sottoproblemi:

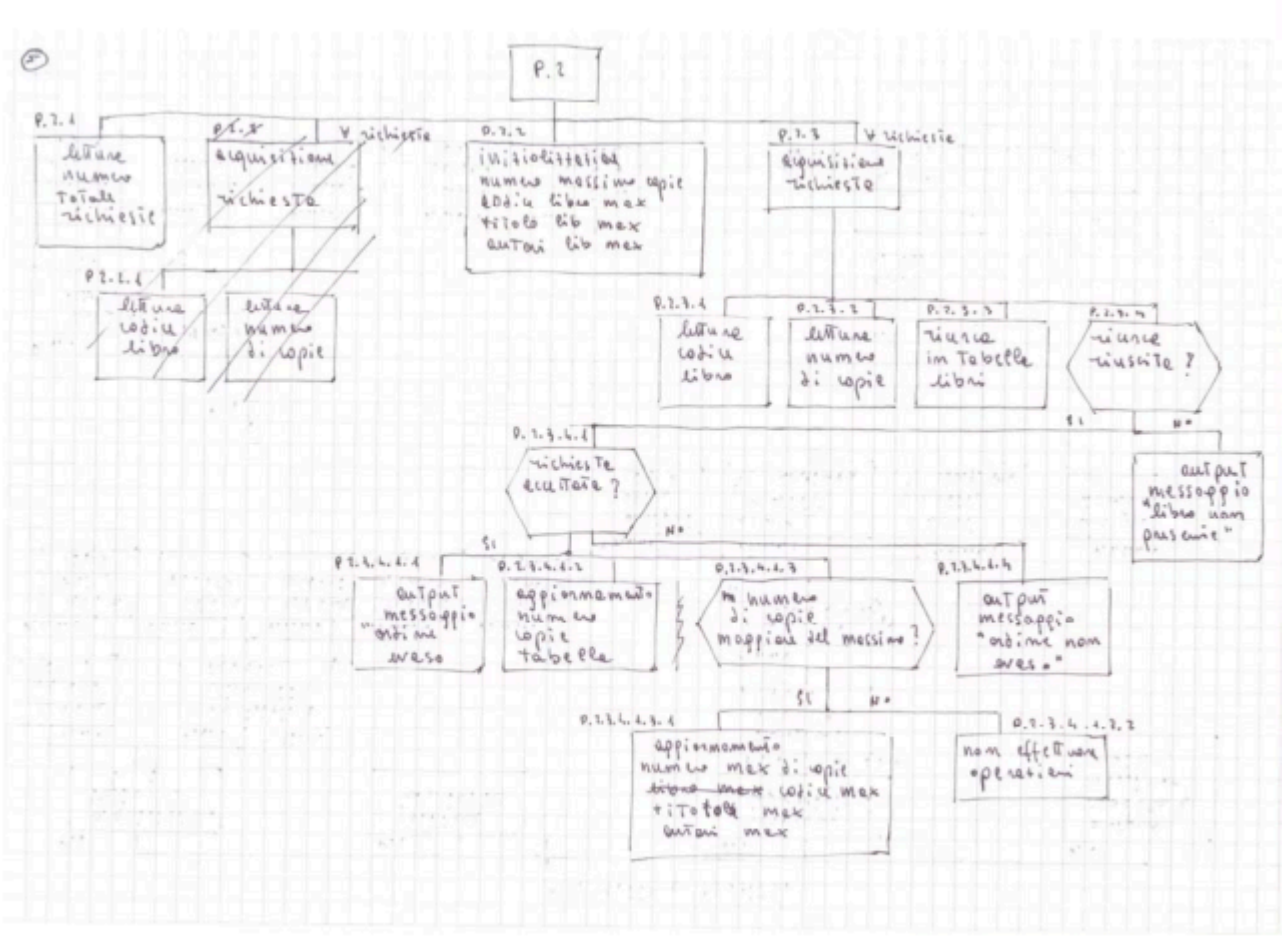
- Gestione Libreria
  - P1: Acquisizione Tabella
    - P1.1: Lettura numero libri
    - P1.2: ∀ Acquisizione libri
      - P1.2.1: Acquisisci codice
      - P1.2.2: Acquisici titolo
      - P1.2.3: Acquisci autore
      - P1.2.4: Lettura disponibile
      - P1.2.5: Lettura scorta
  - P2: Gestire richieste
    - P2.1: Inizializiamo il massimo
    - P2.2: ∀ Acquisizione delle richieste
      - P2.2.1: Acquisire codice libro
      - P2.2.2: Acquisire numero copie

        - P2.2.3: Ricerca libro
        - P2.2.4: (Selezione) Ricerca riuscita
            - P2.2.4.1: (Selezione) Richiesta accettata **SI**
            - P2.2.4.1.1: Aggiornamento massimo
            - P2.2.4.2: Richiesta rifiutata **NO**
- P3: Gestione approvvigionamento
    - P3.1: ∀ Scansione tabella libri
        - P3.1.1: (Selezione) Copie in esaurimento?
        - **SI:**
            - P3.1.1.1: Output codice libro
            - P3.1.1.2: Calcolo quantità da riordinare
            - P3.1.1.3: Output quantità da riordinare
        - **NO**:
            - P3.1.1.4: Null
- P4: Tabella aggiornata
    - P4.1: ∀ Output tabella appionate
        - P4.1.1: Output codice libro
        - P4.1.2: Output titolo
        - P4.1.3: Output autori
        - P4.1.4: Output numero copie
        - P4.1.5: Output numero scorta
    - P4.2: Output copie massimo
        - P4.2.1: Codice libro
        - P4.2.2: Numero max

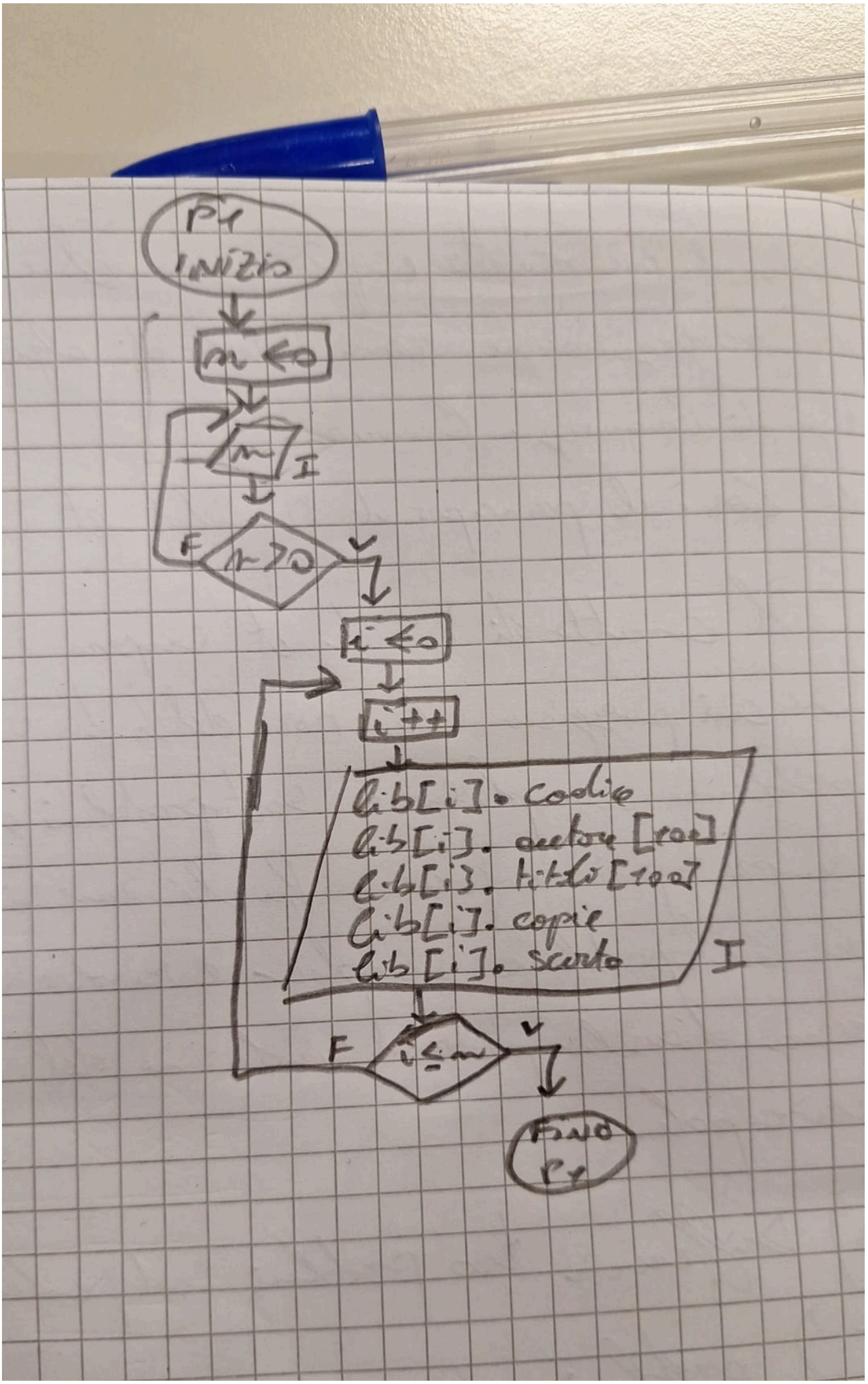
ALBERO GRAFICO

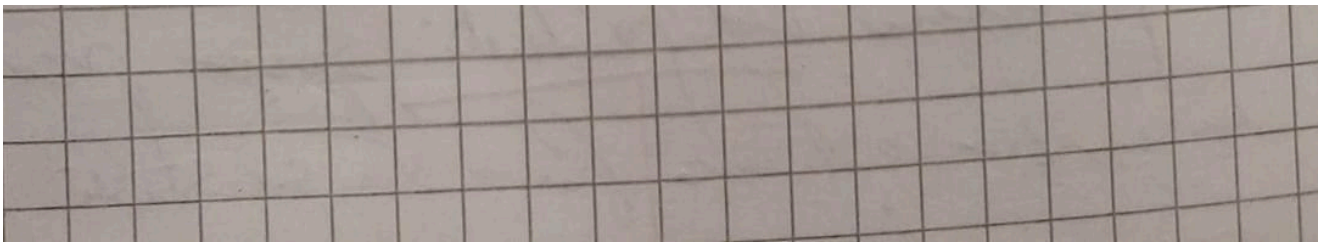


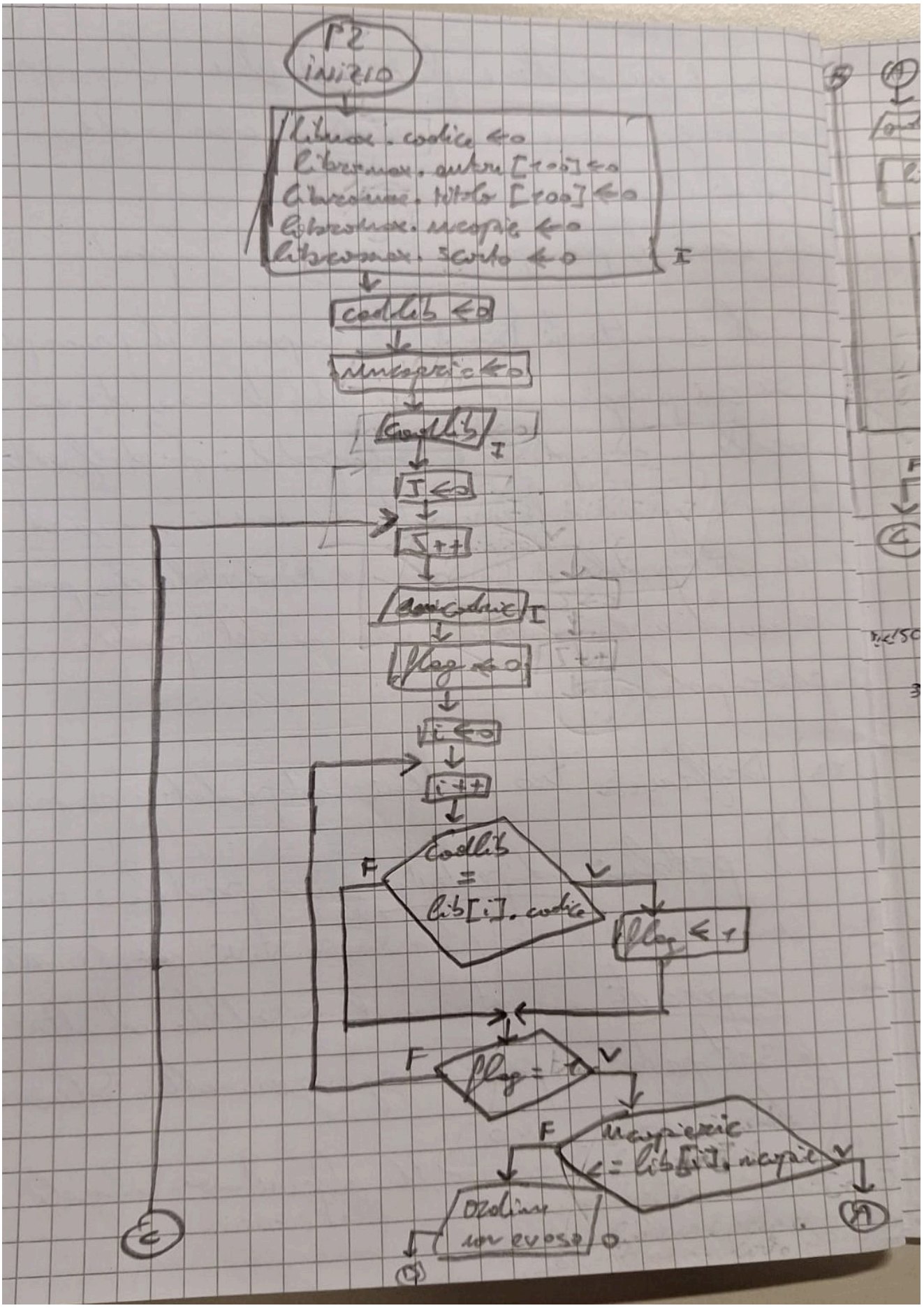

P4
P.4.1
output tabelle aggiornate
∀ libro
P.4.2
output copie massimo
P.4.1.1
output codice libro
P.4.1.2
output titolo
P.4.1.3
output autori
P.4.1.4
output numero copie
P.4.1.5
output numero scorte
codice libro
numero max

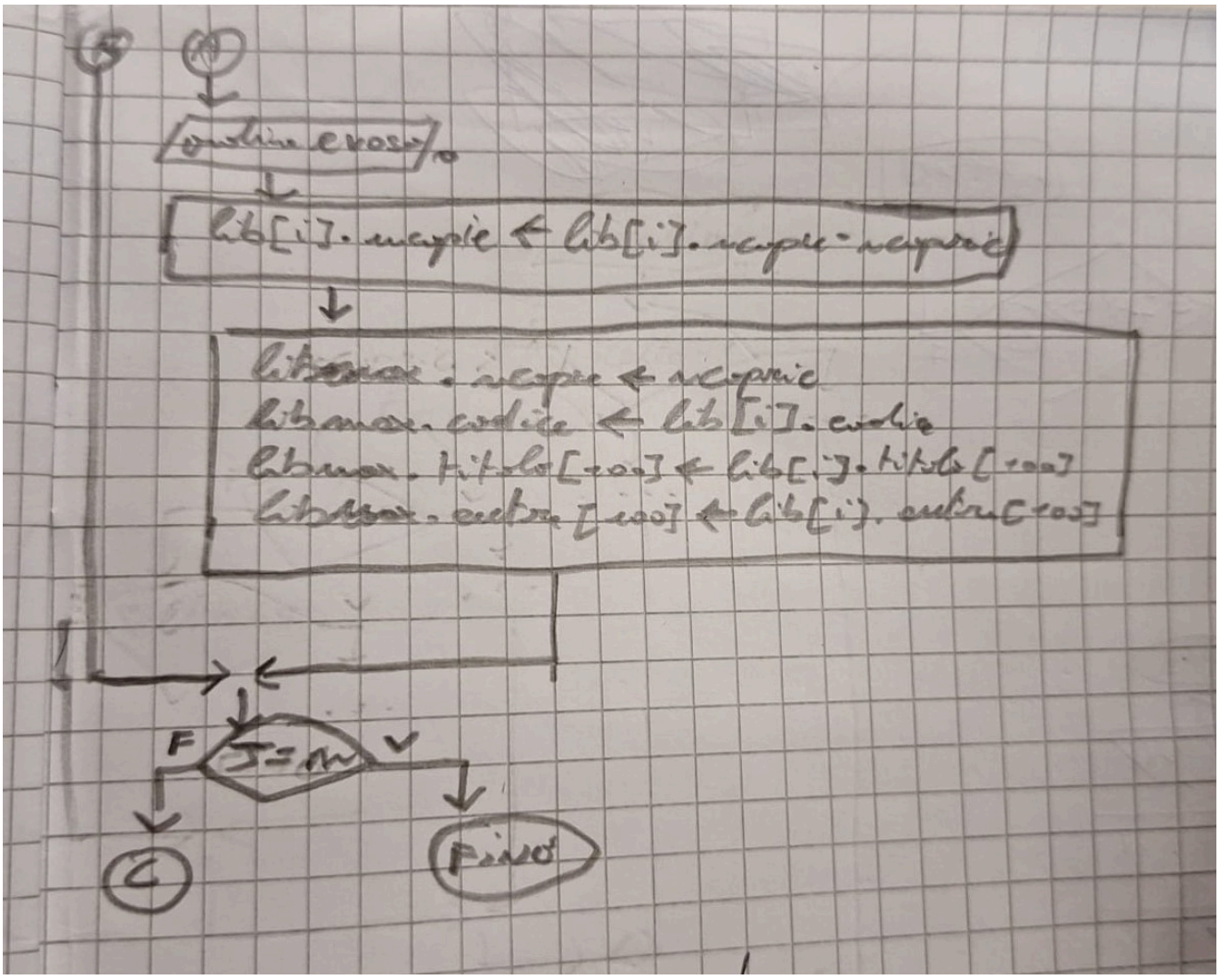

P.2
P.2.1
lettura numero totale richieste
P.2.2
inizializzazione numero massimo copie codice libro max titolo lib max autori lib max
P.2.3
∀ richieste
acquisizione richieste
P.2.3.1
lettura codice libro
P.2.3.2
lettura numero di copie
P.2.3.3
ricerca in tabella libri
P.2.3.4
riuscita ?
SI
NO
output messaggio "libro non presente"
P.2.3.4.1
richiesta eseguita ?
SI
NO
P.2.3.4.1.1
output messaggio "ordine evaso"
P.2.3.4.1.2
aggiornamento numero copie tabella
P.2.3.4.1.3
n numero di copie maggiore del massimo ?
P.2.3.4.1.4
output messaggio "ordine non evaso"
SI
NO
P.2.3.4.1.3.1
aggiornamento numero max di copie codice max titolo max autori max
P.2.3.4.1.3.2
non effettua operazioni

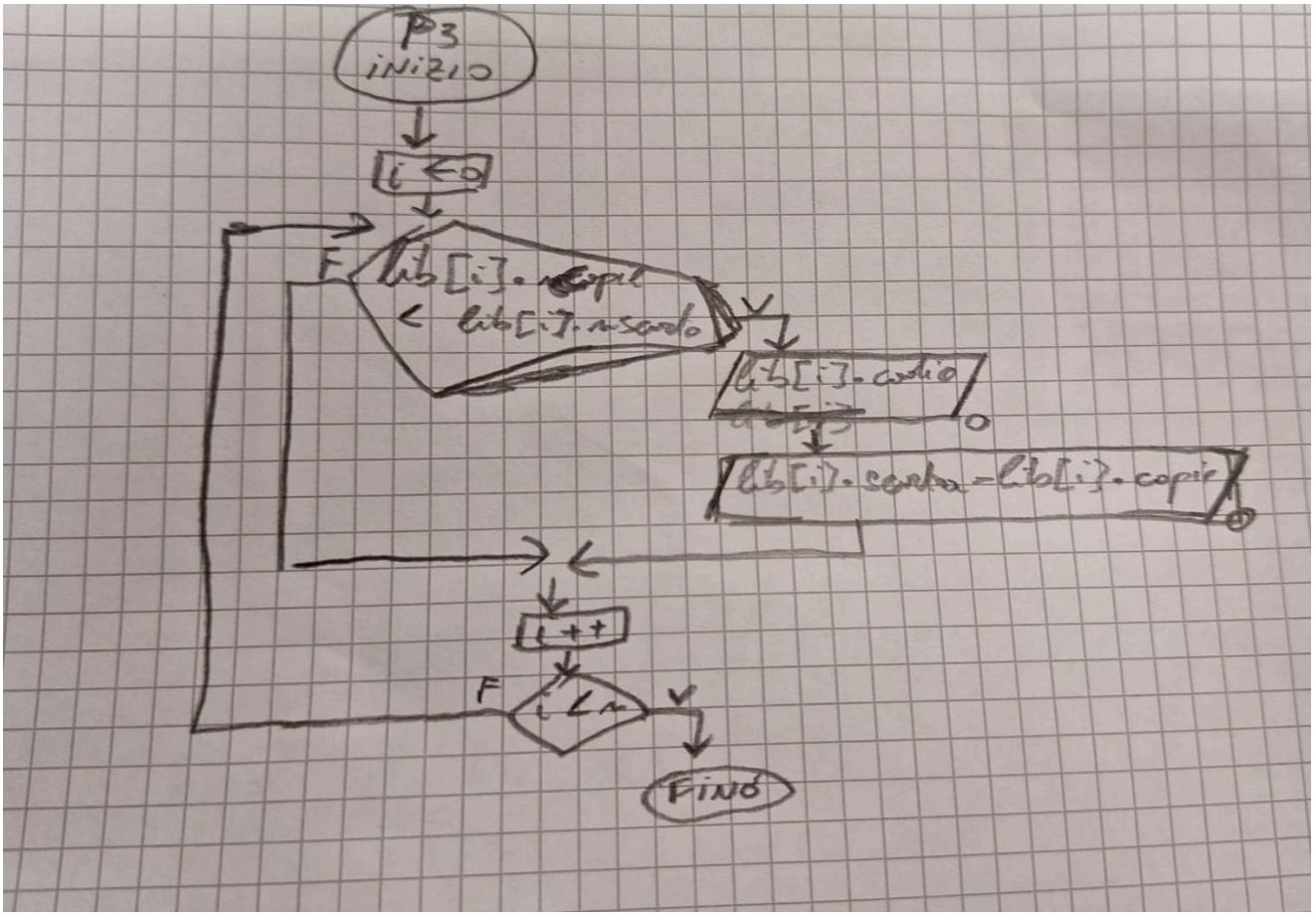
FLOWCHART

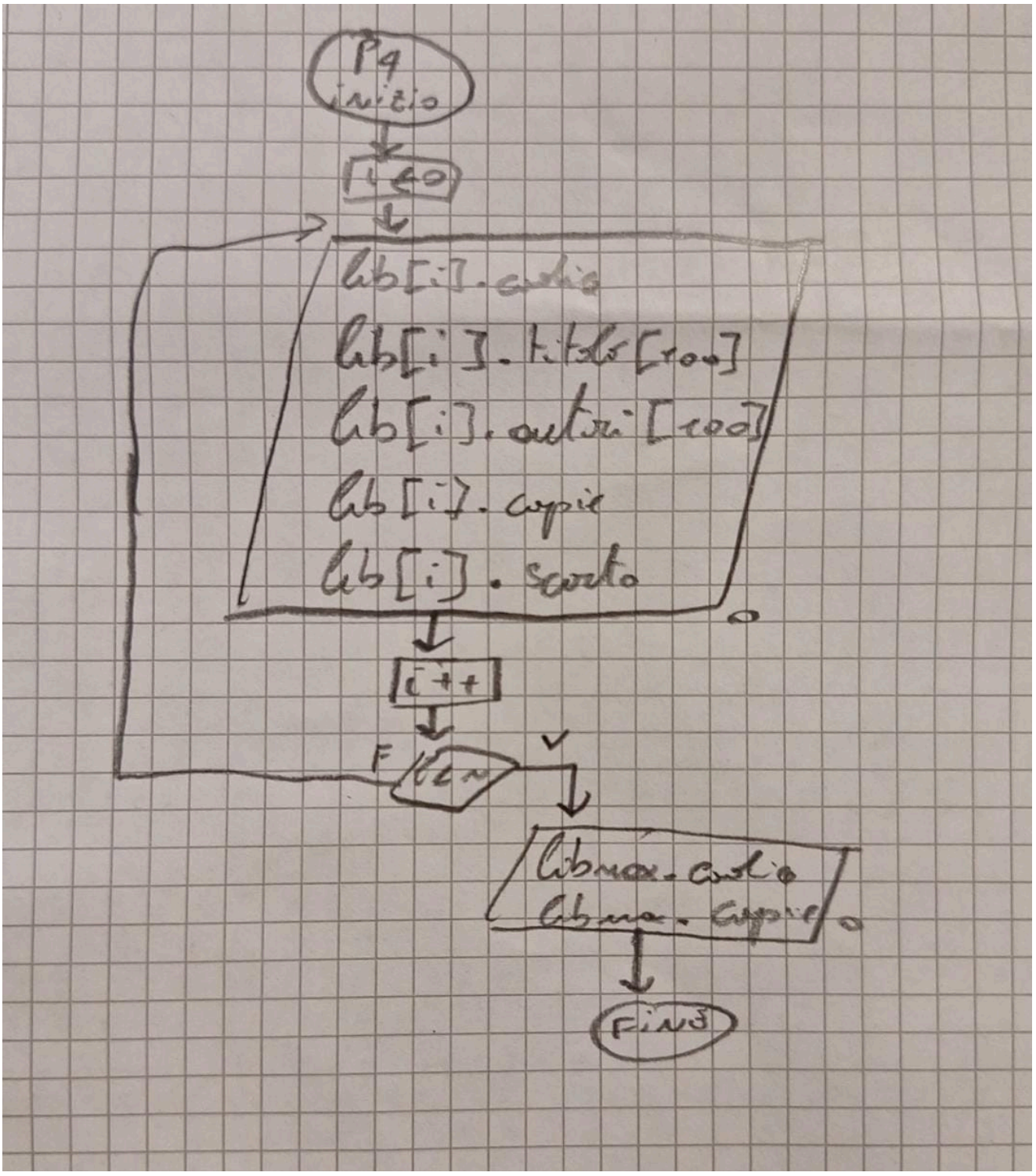

P1
INIZIO
n ← 0
n
I
F
n > 0
V
i ← 0
i++
lib[i].codice
lib[i].autore[100]
lib[i].titolo[100]
lib[i].copie
lib[i].scarto
I
F
i ≤ n
V
FINE
P1




P2
INIZIO
libreria.codice ← 0
libreria.autore[100] ← 0
libreria.titolo[100] ← 0
libreria.nicopie ← 0
libreria.scarto ← 0
I
codlib ← 0
nucopie ← 0
codlib
I
I ← 0
I++
codlibric
I
flag ← 0
i ← 0
i++
codlib = lib[i].codice
F
V
flag ← 1
flag = 1
F
V
nucopieric ← lib[i].nucopie
F
V
Ordine non eseguito
C
A
B


B
A
ordina e rosso
lib[i].ncopie ← lib[i].ncopie - ncopie
libmax.ncopie ← ncopie
libmax.codice ← lib[i].codice
libmax.titolo[100] ← lib[i].titolo[100]
libmax.autore[100] ← lib[i].autore[100]
F
j = n
V
C
FINE


P3
INIZIO
i ← 0
F
lib[i].copie
< lib[i].nscarto
V
lib[i].codice
lib[i].scarto = lib[i].copie
i++
F
i < n
V
FINE



CODICE

```
/*
 ==============================================================================
 Name        : ESERCIZIO.c
 Author      :
 Version     :
 Copyright   : Your copyright notice
 Description : Hello World in C, Ansi-style
 ==============================================================================
 */
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAXLUNGH 50
#define MAXLIBRI 10

typedef struct {
        char codice[MAXLUNGH];
        char titolo[MAXLUNGH];
        char autore[MAXLUNGH];
        int copie;
        int scorta;
}Lib_t;

void acquisisciTabella(Lib_t Libreria [], int dim);
void stampa(Lib_t libro);

int main(void) {

        int n_lib;
        Lib_t Lib[MAXLIBRI];
        Lib_t libMax;
        char codiceRichiesta[MAXLUNGH];
        int copieRichiesta;
        char risp;
        short flag=0, i;



        do {
                printf("\nQuanti libri inserire --> ");
                scanf("%d", &n_lib);
                getchar();
        } while (n_lib<=0 || n_lib >MAXLIBRI);

        acquisisciTabella(Lib, n_lib);

        libMax.copie=-1;
        do {
                printf("\nInserire codice libro da cercare --> ");
                scanf("\n%5s", codiceRichiesta);
```

```
                printf("\nQuanti libri acquistare --> ");
                scanf("%d", &copieRichiesta);
                flag=0;
                i=0;
                do{
                        if(strcmp(codiceRichiesta,Lib[i].codice)==0)
                                flag=1;
                        i++;
                } while (i<n_lib && flag==0);
                i--; // torniamo indietro all'elemento trovato
                if(flag==1){
                        printf("\n Libro trovato!");
                        if (Lib[i].copie >= copieRichiesta){
                                printf("\nOrdine evaso!\n");
                                Lib[i].copie-=copieRichiesta;
                                if (copieRichiesta>libMax.copie)
                                {
                                        libMax=Lib[i];
                                        libMax.copie=copieRichiesta;
                                }
                        }
                        else{
                                printf("\n Ordine non evadibile\n");
                        }
                } else
                        printf("\n libro non trovato");

                printf("Vuoi continuare (s/n)?");
                scanf("\n%c", &risp);
        }while(risp=='s' || risp=='S');

        printf("\n Il libro più richiesto è: ");
        stampa(libMax);

    /*Secondo l'analisi non si gestirà il caso di errore
    in caso avessimo 2 libri con stesso codice o 2 massimi uguali
    per permette la riuscita di un esame rapito ed efficente.*/

        return EXIT_SUCCESS;
}

void acquisisciTabella(Lib_t Libreria [], int dim) {
```

```
        for(int i=0; i<dim;i++){
                printf("\n Inserire i dati del libro: \n");
                printf("\t Codice (solo 5 caratteri)--> ");
                fgets(Libreria[i].codice, sizeof(Libreria[i].codice), stdin);
                //uso di fgets per prevenire gli spazzi tra caratteri nei nomi
dei libri
                Libreria[i].codice[strcspn(Libreria[i].codice, "\n")] = '\0'; //
Rimuove newline

                printf("\t Titolo -->");
                fgets(Libreria[i].titolo, sizeof(Libreria[i].titolo), stdin);
                //uso di fgets per prevenire gli spazzi tra caratteri nei nomi
dei libri
                Libreria[i].titolo[strcspn(Libreria[i].titolo, "\n")] = '\0'; //
Rimuove newline

                printf("\t Autore -->");
                fgets(Libreria[i].autore, sizeof(Libreria[i].autore), stdin);
                //uso di fgets per prevenire gli spazzi tra caratteri nei nomi
dei libri
                Libreria[i].autore[strcspn(Libreria[i].autore, "\n")] = '\0'; //
Rimuove newline

                printf("\t NumCopie -->");
                scanf("%d", &Libreria[i].copie);
                getchar();

                printf("\t Scorta minima -->");
                scanf("%d", &Libreria[i].scorta);
            getchar();
        }
}

void stampa(Lib_t libro) {

        printf("\t Codice --> %s\n", libro.codice);
        printf("\t Titolo --> %s\n", libro.titolo);
        printf("\t Autore --> %s\n", libro.autore);
        printf("\t NumCopie --> %d\n", libro.copie);
        printf("\t Scorta minima -->%d\n", libro.scorta);

}
```

link per visualizzare su GDB il codice